

Graphs in Machine Learning

— Spring Quarter, Lecture 1 —

Prof. Jeff Bilmes

University of Washington, Seattle

Departments of: Electrical & Computer Engineering, and Computer Science & Engineering
<http://melodi.ee.washington.edu/~bilmes>

March 31st, 2025

Announcements

- This is a class on:
 - ① Graphs
 - ② Probabilistic graphical models, and
 - ③ Graph neural networksin machine learning.
- Welcome to the class!
- We have a lot to learn in a short amount of time.
- Today, we'll cover logistics, and then briefly our three main topics.

Class information

- Mon, Wed, 4:00-6:00pm in ECE Rm. 269 (this room).
- Lecturer: Prof. Jeff Bilmes, office hours Wednesday nights, 10:00pm-11:00pm via zoom (link to be posted to canvas).
- Also available online by appointment.
- Graders: Shwet Chitnis <s2chitni@uw.edu> and Haresh Indrajit <haresh@uw.edu>
- On our web page (<https://canvas.uw.edu/courses/1800371>), we will have announcements, readings, homework assignments, copies of these slides, bboard, and so on.
- We also have a discord server: <https://discord.gg/fxQyquA5> (good for 7 days, posted to canvas).
- Copies of lecture slides available on the web before lecture.
- Videos of lectures available on youtube soon after each lecture.
- Readings will be these slides + papers available for free on the web.

Grading

- **(A)** One homework assignment approximately every 2-2.5 weeks, each a combination of some math and programming (must be in python/python libraries).
- **(B)** Online canvas quizzes roughly every 1.5 weeks, will overlap other assignments.
- **(C)** In-class **closed book/notes/device** multiple choice quizzes, every other Wednesday for the first 20 minutes of class (starting Wed, April 9th).
- **(D)** Final project (paper reading + implementation + testing on graphical models or graph NNs). 4 page writeup + 5 minute youtube video (played on final exam day).
- No other in-class tests or final exam (final quizzes **will** occur during final exam week).
- Grades based on: **(A)** Homeworks: 22.5%. **(B)** Canvas Quizzes: 20%. **(C)** In-class quizzes: 35%. **(D)** Final Project: 20%. **(E)** Perceived engagement: 2.5% (all or nothing).
- “Perceived engagement” corresponds to engagement, which means not only attendance but also are you engaged (e.g., ask and/or answer questions on canvas/discord, involved, quality, etc.).
- HW grading will be based entirely on quality, completeness, clarity, following instructions, and correctness in your HW problem sets and python ML projects.
- Final grade will be on a curve.

More on Assignments

- Problem sets: answer a question, prove a simple mathematical theorem, etc.
- Programming projects, must be familiar with python & python libraries! (we do not plan to offer python tutorials, so if you are rusty please practice up).
- Homework is accepted only electronically in PDF and Zip form via canvas at our assignment dropbox (<https://canvas.uw.edu/courses/1800371/assignments>).
- **Always submit two files:**
 - 1 PDF file for any textual problem sets, math problems, figures, plots, graphs, and other graphics, derivations worked out by hand on paper (scanned and converted to PDF).
 - 2 Use a second .zip file for anything else (code, data, .ipynb files, etc.).
 - 3 We may experiment with gradescope (TBD).

PLEASE READ: Homework/Online-Quiz late policy

- For every additional day you take for the submission, we would subtract 5% of your grade.
- This continues for a week, after which we will stop accepting submissions and release the solutions.
- Example: $\text{EffectiveGrade} = \text{OriginalGrade} \times (1 - \text{LateDays} \times 0.05)$ where $0 \leq \text{LateDays} \leq 7$.
- If the homework is due on the 6th and you submit on the morning of 7th, you'll get max 95% of your grade, and so on up till 11:59 on the 13th – your last day to submit – when you will get max 65% of your grade.
- We recommend that you submit however much you have completed by the deadline.
- If you resubmit your assignment after the deadline, please make note of which problem(s) you have updated; we will only apply the reduction to the relevant problems (problem granularity, not subproblem granularity).
- There will be **NO late days** for the last homework, since there would be a very short window for us to grade it.

On Final Project

- Final project, 4-page conference-style paper, and a 5-minute final presentation video (uploaded to youtube with the link posted to canvas).
- Milestones (1-page project proposal, 1-page progress summaries) in the weeks before final project. These will be graded.
- Final project should be about graphical models and/or graph neural networks - e.g., used in an application, as a researcher (i.e., new inference method, new proof, new application), or can be a review of a set of papers (that we do not cover).
- The date of the final project presentation is Friday, June 13th, 2025, our room (ECE Rm. 269).
- All final project related assignments turned in electronically via our assignment dropbox (<https://canvas.uw.edu/courses/1800371/assignments>) (unless we move to gradescope).
- More on this later this quarter.

Recommended Readings

- Arumugam et. al. “Handbook of Graph Theory, Combinatorial Optimization, and Algorithms”, 2016
- *Algorithms and Data Structures for Exact Computation of Marginals*. In Marloes Maathuis, Mathias Drton, Steffen Lauritzen and Martin Wainwright, editors, Handbook of Graphical Models, pp. 83-110, Chapman & Hall/CRC Handbooks of Modern Statistical Methods, 2018.
- Wainwright and Jordan *Graphical Models, Exponential Families, and Variational Inference*
<http://www.nowpublishers.com/product.aspx?product=MAL&doi=22000000001>
- Hamilton, “Graph Representation Learning”, 2021
- Other paper readings will be mentioned as we proceed.

Class Road Map/Progress - EE512 - Spring 2025

- L1 (3/31): Introduction, Overview: Graphs, Graphical Models, Graph Neural Networks
- L2 (4/2):
- L3 (4/7):
- L4 (4/9):
- L5 (4/14):
- L6 (4/16):
- L7 (4/21):
- L8 (4/23):
- L9 (4/28):
- L10 (4/30):
- L11 (5/5):
- L12 (5/7):
- L13 (5/12):
- L14 (5/14):
- L15 (5/19):
- L16 (5/21):
- LXX (6/26): Memorial day holiday
- L17 (5/28):
- L18 (6/2):
- L19 (6/4):
- Final Presentations due: (Friday 6/13/2023):

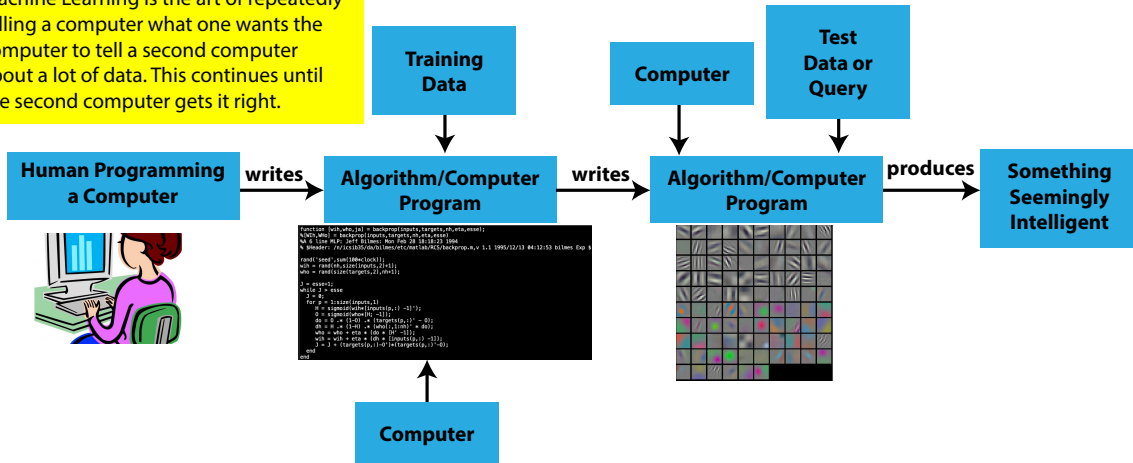
Holidays: Monday 6/26 (Memorial day). Last day of instruction: Fri, 6/7. Final Exam Week: 6/7-6/13. Our Exam Time: Fri, June 13th, 2:30 - 4:30 PM, ECE 267.

The Class Concerto

- ① 25%: Graphs and Graphs in Machine Learning (most generally).
- ② 37.5%: Probabilistic Graphical Models - the use of graphs in graphical models, probabilistic graphical model inference, computing probabilities either exactly or approximately, and applications in machine learning.
- ③ 37.5%: Graph neural networks - using graphs to structure and train deep neural networks.

Machine Learning to achieve AI

Machine Learning is the art of repeatedly telling a computer what one wants the computer to tell a second computer about a lot of data. This continues until the second computer gets it right.



Machine-Learning can be seen as a form of “Indirect Algorithms”: so far, the best way we know to get computers to practically achieve what is called “artificial intelligence” (AI).

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- There is some family of models $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, that for any θ (parameters) do a mapping.

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- There is some family of models $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, that for any θ (parameters) do a mapping.
- Goal of supervised machine learning is to find the θ that minimizes average regularized loss.

$$\theta^* \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(h_\theta(x^{(i)}), y^{(i)}) \quad (1.1)$$

where Θ is the set of allowable values of θ , $\mathcal{L}(\theta)$ is overall loss for θ on $\mathcal{D}$, $\Omega(\theta)$ is an (explicit) complexity penalty regularizer (Occam's razor), λ a tradeoff parameter.

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- There is some family of models $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, that for any θ (parameters) do a mapping.
- Goal of supervised machine learning is to find the θ that minimizes average regularized loss.

$$\theta^* \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(h_\theta(x^{(i)}), y^{(i)}) \quad (1.1)$$

where Θ is the set of allowable values of θ , $\mathcal{L}(\theta)$ is overall loss for θ on $\mathcal{D}$, $\Omega(\theta)$ is an (explicit) complexity penalty regularizer (Occam's razor), λ a tradeoff parameter.

- And then once done, hope (show, prove) that learnt model works well on unseen data.

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- There is some family of models $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, that for any θ (parameters) do a mapping.
- Goal of supervised machine learning is to find the θ that minimizes average regularized loss.

$$\theta^* \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(h_\theta(x^{(i)}), y^{(i)}) \quad (1.1)$$

where Θ is the set of allowable values of θ , $\mathcal{L}(\theta)$ is overall loss for θ on $\mathcal{D}$, $\Omega(\theta)$ is an (explicit) complexity penalty regularizer (Occam's razor), λ a tradeoff parameter.

- And then once done, hope (show, prove) that learnt model works well on unseen data.
- Many variants, θ could be a real-valued vector, set of vectors, set of matrices, set of discrete structures, and combinations thereof. Many choices: algorithmic, data, inductive biases, computational vs. memory tradeoffs, etc.

Supervised Machine Learning: Generalizing Associative Memories

- We are given a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- There is some family of models $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, that for any θ (parameters) do a mapping.
- Goal of supervised machine learning is to find the θ that minimizes average regularized loss.

$$\theta^* \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(h_\theta(x^{(i)}), y^{(i)}) \quad (1.1)$$

where Θ is the set of allowable values of θ , $\mathcal{L}(\theta)$ is overall loss for θ on $\mathcal{D}$, $\Omega(\theta)$ is an (explicit) complexity penalty regularizer (Occam's razor), λ a tradeoff parameter.

- And then once done, hope (show, prove) that learnt model works well on unseen data.
- Many variants, θ could be a real-valued vector, set of vectors, set of matrices, set of discrete structures, and combinations thereof. Many choices: algorithmic, data, inductive biases, computational vs. memory tradeoffs, etc.
- This is all the topic of ECE-511 (which we offered in Winter 2025) and, in this class, we assume some knowledge of.

Supervised Machine Learning: Training

- Many many ways to do this, but common is to do (incremental or stochastic) gradient descent, where we adjust θ to minimize overall loss.

Algorithm 1: General Gradient Descent (Batch, Stochastic, or Mini-Batch)

Input : Initial parameters θ_0 , learning rate schedule η_t , loss function $\mathcal{L}(\theta)$

Output: Optimized parameters θ^*

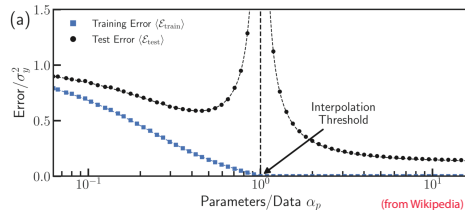
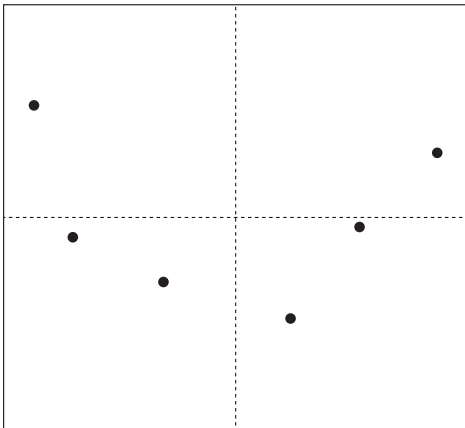
```

1   $t \leftarrow 0$ 
2  while not converged do
3      if Batch Gradient Descent then
4           $D_t \leftarrow$  entire dataset
5      else
6          if Stochastic Gradient Descent then
7               $D_t \leftarrow$  a single randomly selected data point
8          else
9              Mini-Batch Gradient Descent:
10              $D_t \leftarrow$  random subset of data (mini-batch)
11         end
12     end
13     Compute gradient estimate:  $g_t \approx \nabla_{\theta} \mathcal{L}_{D_t}(\theta_t)$ 
14     Update parameters:  $\theta_{t+1} \leftarrow \theta_t - \eta_t \cdot g_t$ 
15      $t \leftarrow t + 1$ 
16 end
17 return  $\theta_t$ 

```

Supervised Machine Learning: Overfitting, Underfitting, Double Descent

- Overfitting, Underfitting, Bias, Variance, Inductive Bias, Double Descent & Overparameterization



Supervised Machine Learning: Models

- Many models:
 - Logistic & Softmax Regression, Decision Tree Ensembles, Nearest Neighbor Classifiers, Kernel Machines, Neural Networks
 - Sequence models: Hidden Markov models & Kalman Filters, Conditional Random Fields, Recurrent Neural Networks

Supervised Machine Learning: Models

- Many models:
 - Logistic & Softmax Regression, Decision Tree Ensembles, Nearest Neighbor Classifiers, Kernel Machines, Neural Networks
 - Sequence models: Hidden Markov models & Kalman Filters, Conditional Random Fields, Recurrent Neural Networks
- In this class, we want to see how graphs are used in machine learning in particular:
 - Graphical Models and (a bit of) Dynamic Graphical Models
 - Graph Neural Networks (GNNs).

Graphs

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.
- More formally, we say $G = (V, E)$ where V is a size $|V| = n$ set nodes (w.l.o.g., we say $V = \{0, 1, 2, \dots, n - 1\}$ and $E \subseteq V \times V$ is a set of pairs of nodes.

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.
- More formally, we say $G = (V, E)$ where V is a size $|V| = n$ set nodes (w.l.o.g., we say $V = \{0, 1, 2, \dots, n - 1\}$ and $E \subseteq V \times V$ is a set of pairs of nodes.
- Each $v \in V$ is a node. Each $e \in E$ is a pair, e.g., $e = \{v_1, v_2\}$ or $e = (v_1, v_2)$ where $v_1, v_2 \in V$.

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.
- More formally, we say $G = (V, E)$ where V is a size $|V| = n$ set nodes (w.l.o.g., we say $V = \{0, 1, 2, \dots, n-1\}$ and $E \subseteq V \times V$ is a set of pairs of nodes.
- Each $v \in V$ is a node. Each $e \in E$ is a pair, e.g., $e = \{v_1, v_2\}$ or $e = (v_1, v_2)$ where $v_1, v_2 \in V$.
- Graphs are often weighted, with a (usually non-negative) weight function $w : E \rightarrow \mathbb{R}_+$ that maps from edges to real-values. Then graph becomes $G = (V, E, w)$.

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.
- More formally, we say $G = (V, E)$ where V is a size $|V| = n$ set nodes (w.l.o.g., we say $V = \{0, 1, 2, \dots, n-1\}$ and $E \subseteq V \times V$ is a set of pairs of nodes.
- Each $v \in V$ is a node. Each $e \in E$ is a pair, e.g., $e = \{v_1, v_2\}$ or $e = (v_1, v_2)$ where $v_1, v_2 \in V$.
- Graphs are often weighted, with a (usually non-negative) weight function $w : E \rightarrow \mathbb{R}_+$ that maps from edges to real-values. Then graph becomes $G = (V, E, w)$.
- Still more generally, any node $v \in V$, edge $e \in E$, or group of nodes or group of edges, can have an associated set of attributes or properties or features (e.g., numbers, vectors, text, image, relation types on edges, or any other useful bits of information as input or to be predicted as output).

Graphs

- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.
- More formally, we say $G = (V, E)$ where V is a size $|V| = n$ set nodes (w.l.o.g., we say $V = \{0, 1, 2, \dots, n-1\}$ and $E \subseteq V \times V$ is a set of pairs of nodes.
- Each $v \in V$ is a node. Each $e \in E$ is a pair, e.g., $e = \{v_1, v_2\}$ or $e = (v_1, v_2)$ where $v_1, v_2 \in V$.
- Graphs are often weighted, with a (usually non-negative) weight function $w : E \rightarrow \mathbb{R}_+$ that maps from edges to real-values. Then graph becomes $G = (V, E, w)$.
- Still more generally, any node $v \in V$, edge $e \in E$, or group of nodes or group of edges, can have an associated set of attributes or properties or features (e.g., numbers, vectors, text, image, relation types on edges, or any other useful bits of information as input or to be predicted as output).
- Graphs are used for many purposes, one reason is that they can be quite visual and intuitive.

Why Graphs (and why Graphs in ML)?

- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.

Why Graphs (and why Graphs in ML)?

- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.
- Graphs naturally represent **relational** data, **relations**, and other structures found in social, biological, knowledge, and many other (both natural and human-created) systems.

Why Graphs (and why Graphs in ML)?

- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.
- Graphs naturally represent **relational** data, **relations**, and other structures found in social, biological, knowledge, and many other (both natural and human-created) systems.
- Graph-based algorithms inspire and enable efficient reasoning, search, optimization, and inference over structured and unstructured domains.

Why Graphs (and why Graphs in ML)?

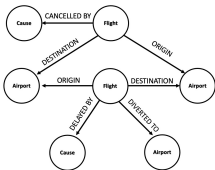
- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.
- Graphs naturally represent **relational** data, **relations**, and other structures found in social, biological, knowledge, and many other (both natural and human-created) systems.
- Graph-based algorithms inspire and enable efficient reasoning, search, optimization, and inference over structured and unstructured domains.
- Graphs are used for complex expressive models like Probabilistic Graphical Models (enabling computation of probabilities exactly or approximately over many collections of related random variables) and Graph Neural Networks (enabling deep learning on non-Euclidean data).

Why Graphs (and why Graphs in ML)?

- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.
- Graphs naturally represent **relational** data, **relations**, and other structures found in social, biological, knowledge, and many other (both natural and human-created) systems.
- Graph-based algorithms inspire and enable efficient reasoning, search, optimization, and inference over structured and unstructured domains.
- Graphs are used for complex expressive models like Probabilistic Graphical Models (enabling computation of probabilities exactly or approximately over many collections of related random variables) and Graph Neural Networks (enabling deep learning on non-Euclidean data).
- Since graphs are visual, lets look at a few ...

Graphs Are Visual (1 of 4)

Slide from: J Leskovic's course.



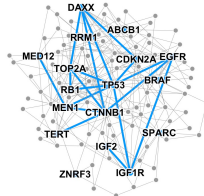
Event Graphs

<https://neo4j.com/blog/developer/the-first-graphgpt-challenge-completed/>

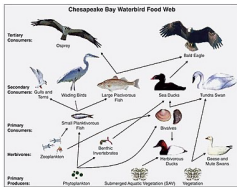


Computer Networks

From: SalientNetworks



Disease Pathways



Food Webs

From: Wikipedia



Particle Networks

From: Pinterest



Underground Networks

From: visitlondon.com

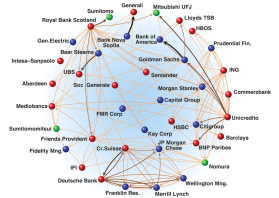
Graphs Are Visual (2 of 4)

Slide from: J Leskovic's course.



Social Networks

From: Medium



Economic Networks

From: Science



Communication Networks

From: Lumen Learning

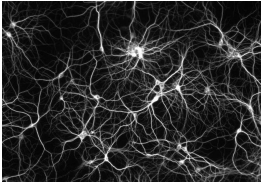


Citation Networks



The Internet Itself

From: Missoula Current News

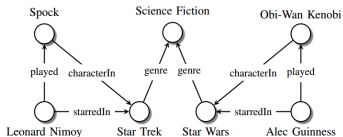


Biological Neural Networks

From: The Conversation

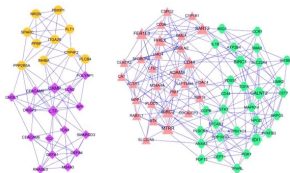
Graphs Are Visual (3 of 4)

Slide from: J Leskovic's course.



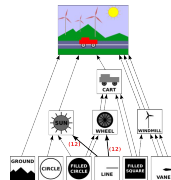
Knowledge Graphs

From: Maximilian Nickel et al



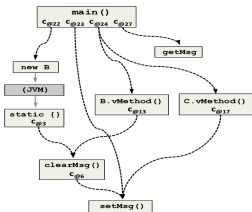
Regulatory Networks

From: ess.wustl.edu



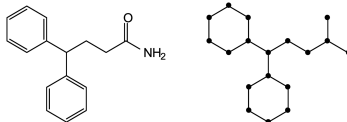
Scene Graphs

From: math.hws.edu



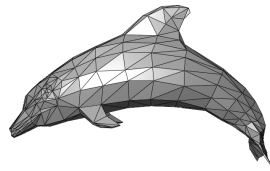
Code/Control Graphs

From: ResearchGate



Molecules

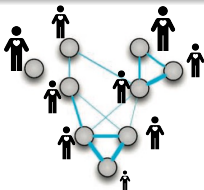
From: MDPI



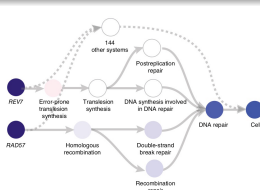
Multi-Dimensional Shapes

From: Wikipedia

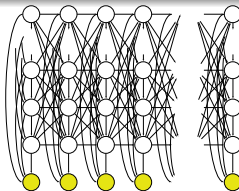
Graphs Are Visual (4 of 4)



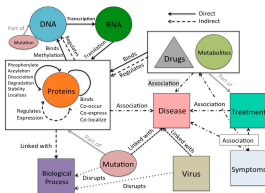
Patient Networks



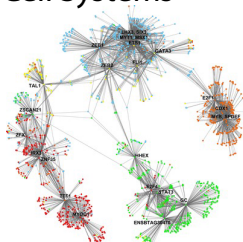
Hierarchies of Cell Systems



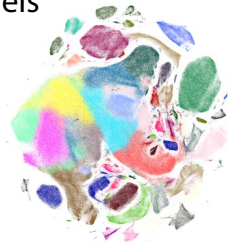
Probabilistic Graphical Models



Biomedical Knowledge Graphs



Gene Interaction Networks



Cell-Cell Similarity Networks

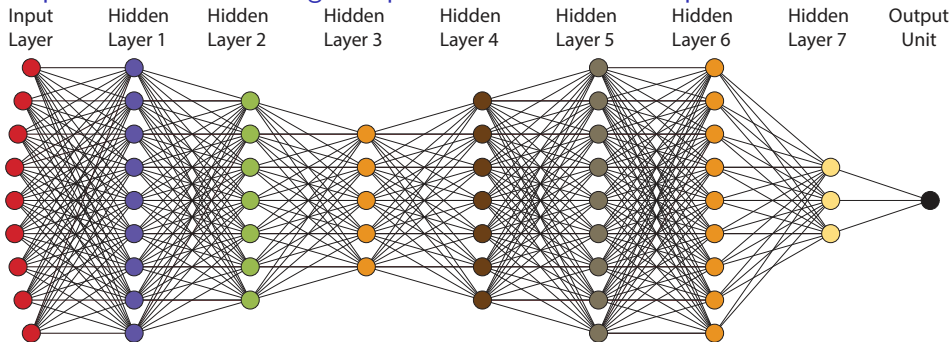
Slide from: Marinka Zitnik's course.

Graphs can describe (neural) data flow

- Graphs can be used to flow of data through a network.

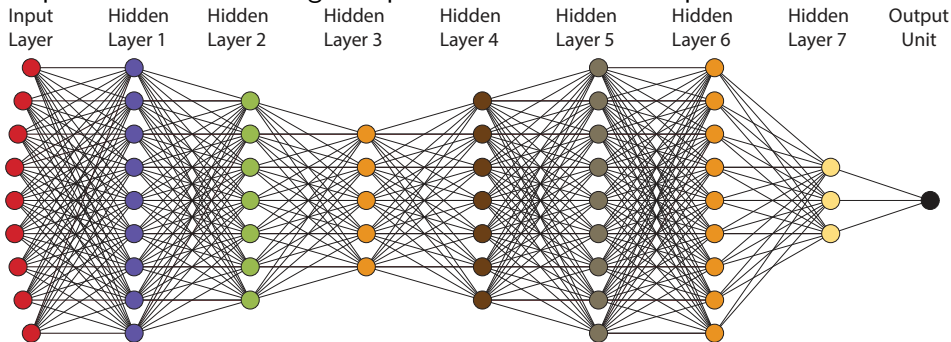
Graphs can describe (neural) data flow

- Graphs can be used to flow of data through a network.
- Classic example in machine learning. Graphs used to describe deep neural networks.



Graphs can describe (neural) data flow

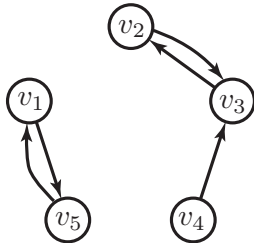
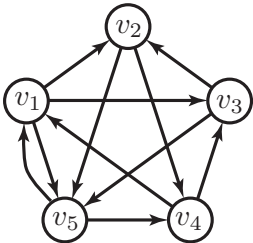
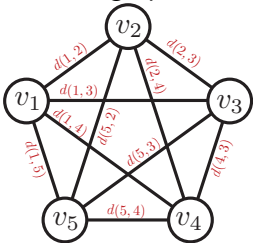
- Graphs can be used to flow of data through a network.
- Classic example in machine learning. Graphs used to describe deep neural networks.



- Graph shows connectivity pattern, and series of matrix-vector multiplications followed by non-linearities.

k -NN Graph

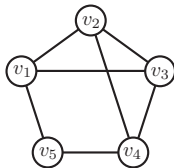
- Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ be a training set and let $d : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}$ be a distance measure (e.g., $d(x, x') = \|x - x'\|$).
- We can represent the distances as a weighted graph. Ex: $n = 5$, fully connected graph **clique**. 2-NN graph. 1-NN graph.



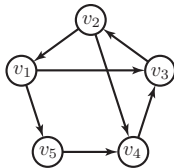
- Note, k -NN graph need not be symmetric. Often, we **symmetrize** the k -NN graph, so some nodes might have more than k **incident edges**.

Matrix representation of graphs

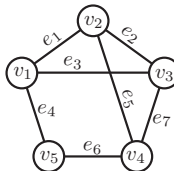
- **adjacency matrix:** $n \times n$ binary matrix indicating if edge $\{u, v\} \in E$, where $n = |V|$ and $G = (V, E)$. Undirected graph, symmetric matrix.
- Adjacency matrix, directed graph, potentially asymmetric matrix.
- **Incidence matrix.** $n \times m$ matrix of nodes/edges with $n = |V|$ and $m = |E|$.



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$



$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$



$$\begin{pmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.
- Sparse matrix representations: Compressed sparse row (CSR) or Compressed sparse column (CSC) or “triplet” Coordinate list (COO) format + several others (many in python <https://docs.scipy.org/doc/scipy/reference/sparse.html>)

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.
- Sparse matrix representations: Compressed sparse row (CSR) or Compressed sparse column (CSC) or “triplet” Coordinate list (COO) format + several others (many in python <https://docs.scipy.org/doc/scipy/reference/sparse.html>)
- Other good representations of/data structures for graphs include:

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.
- Sparse matrix representations: Compressed sparse row (CSR) or Compressed sparse column (CSC) or “triplet” Coordinate list (COO) format + several others (many in python <https://docs.scipy.org/doc/scipy/reference/sparse.html>)
- Other good representations of/data structures for graphs include:
 - Adjacency list: a map from nodes to a list of edges (good for sparse graphs not good for very dense graphs, similar to CSR and CSC matrix representations)

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.
- Sparse matrix representations: Compressed sparse row (CSR) or Compressed sparse column (CSC) or “triplet” Coordinate list (COO) format + several others (many in python <https://docs.scipy.org/doc/scipy/reference/sparse.html>)
- Other good representations of/data structures for graphs include:
 - Adjacency list: a map from nodes to a list of edges (good for sparse graphs not good for very dense graphs, similar to CSR and CSC matrix representations)
 - Edge list: simply a list of edges, list of pairs of nodes (typically unordered). Also good for sparse (but not dense) graphs, fast to iterate over edges, but edge lookup is linear in $|E|$.

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).
- Best representation depends on graph sparsity — in matrix terms, “nnzs” or number of non-zero edges.
- Dense matrix representations of a graph simply use an $n \times n$ or $n \times m$ matrix as the graph.
- Sparse matrix representations: Compressed sparse row (CSR) or Compressed sparse column (CSC) or “triplet” Coordinate list (COO) format + several others (many in python <https://docs.scipy.org/doc/scipy/reference/sparse.html>)
- Other good representations of/data structures for graphs include:
 - Adjacency list: a map from nodes to a list of edges (good for sparse graphs not good for very dense graphs, similar to CSR and CSC matrix representations)
 - Edge list: simply a list of edges, list of pairs of nodes (typically unordered). Also good for sparse (but not dense) graphs, fast to iterate over edges, but edge lookup is linear in $|E|$.
- Which graph representation is best depends on the applications and required algorithms.

Many Types of Graphs

- Undirected Graphs: Edges $\{u, v\}$ are unordered representing symmetric relationships.
- Directed Graphs (Digraphs): Edges (u, v) are directed, representing asymmetric relationships.
- Cliques: Every pair of nodes is connected; a complete graph.
- Trees: Connected acyclic graphs; exactly one path between any two nodes.
- Chains (or Paths): Nodes connected in a single line; a special type of tree.
- Cycles/Circuits: Paths where the first and last nodes are the same (i.e., a closed loop).
- Directed Acyclic Graphs (DAGS): Edges (u, v) have directions and no directed cycles.
- Grids: Nodes in “lattice” structure (2D or higher); common in images and spatial domains.
- Planar Graphs: Can be drawn in 2D without any edges crossing.
- Bipartite Graphs: Nodes can be split into two sets with edges only across sets (not within).
- Multiple Connected Components: Graphs with two or more disjoint connected subgraphs.
- Weighted Graphs: Edges carry numerical values, such as distances or costs.
- Hypergraphs: edges (hyperedges) can connect more than two nodes.
- Dynamic Graphs: model and/or change over time (nodes/edges can appear/disappear).
- Random Graphs: Generated probabilistically, e.g., Erdős-Rényi graphs.
- Expander Graphs: Sparse graphs with strong connectivity (non-separability) properties.
- Small-World: Most nodes not neighbors, but reachable in few steps (social networks).
- Scale-Free: Node degree follows a power law; few hubs dominate connectivity.

Constructing Graphs

- Sometimes graphs naturally occur (e.g., the web, road networks, social networks, citation graphs) or can be observed (e.g., protein-protein interaction networks, non-independence properties between random variables).

Constructing Graphs

- Sometimes graphs naturally occur (e.g., the web, road networks, social networks, citation graphs) or can be observed (e.g., protein-protein interaction networks, non-independence properties between random variables).
- At other times, we must construct the graph (sparse graph over data records or samples, relationship graph over data items) $\Rightarrow$ hyperparameters (e.g., k in k -NN).

Constructing Graphs

- Sometimes graphs naturally occur (e.g., the web, road networks, social networks, citation graphs) or can be observed (e.g., protein-protein interaction networks, non-independence properties between random variables).
- At other times, we must construct the graph (sparse graph over data records or samples, relationship graph over data items) $\Rightarrow$ hyperparameters (e.g., k in k -NN).
- The graph structure is critical for the successful use of the graph.

Graph Representations of Manifolds

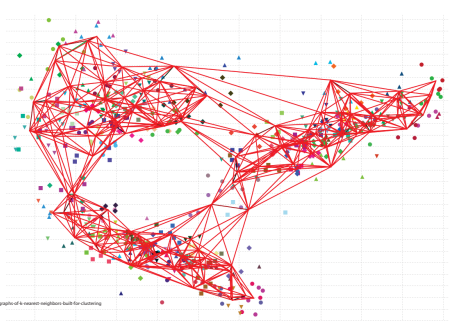
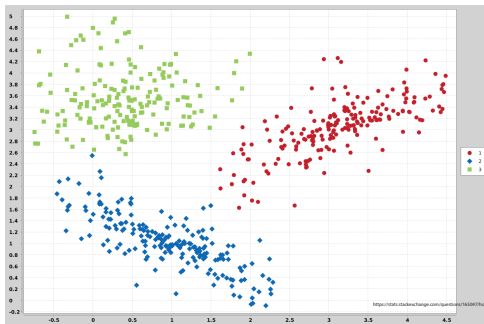
- Sparse k -NN graphs are good at computationally efficiently approximating a low-dimensional manifold, helps to tame the curse of dimensionality issue.

Graph Representations of Manifolds

- Sparse k -NN graphs are good at computationally efficiently approximating a low-dimensional manifold, helps to tame the curse of dimensionality issue.
- Left: Data, Right: k -NN graph from the data. Nearest points determined based on **geodesic distance** (closest along the graph).

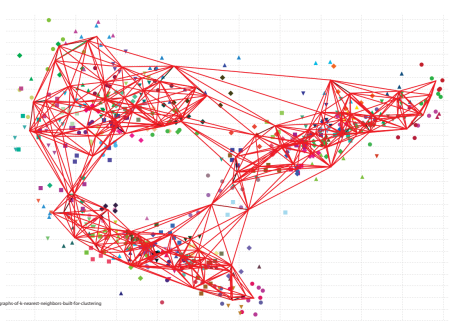
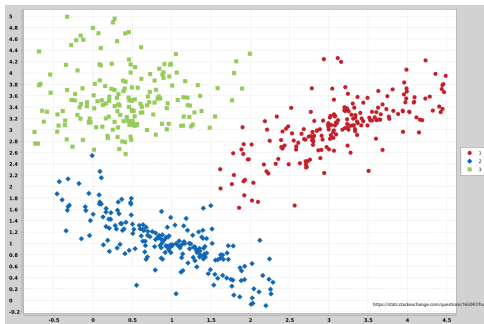
Graph Representations of Manifolds

- Sparse k -NN graphs are good at computationally efficiently approximating a low-dimensional manifold, helps to tame the curse of dimensionality issue.
- Left: Data, Right: k -NN graph from the data. Nearest points determined based on **geodesic distance** (closest along the graph).



Graph Representations of Manifolds

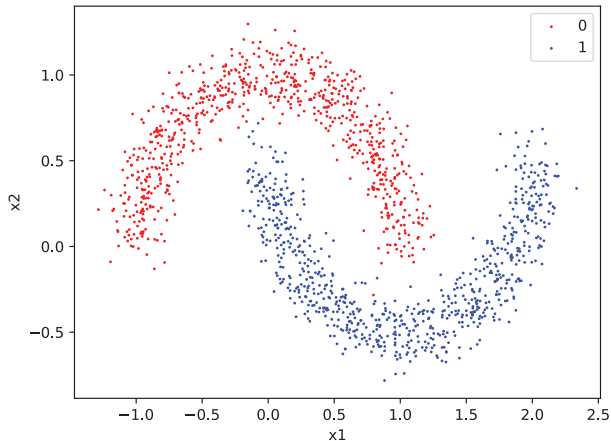
- Sparse k -NN graphs are good at computationally efficiently approximating a low-dimensional manifold, helps to tame the curse of dimensionality issue.
- Left: Data, Right: k -NN graph from the data. Nearest points determined based on **geodesic distance** (closest along the graph).



- Getting the right value for k crucial for proper representation.

Classic Two Moons Dataset

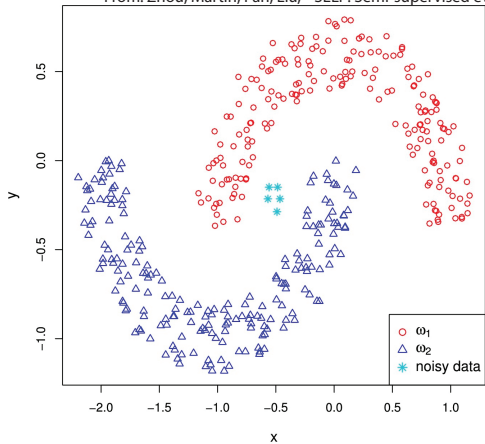
Good example where a graph can represent the two 1D manifolds for each class embedded in 2D space.



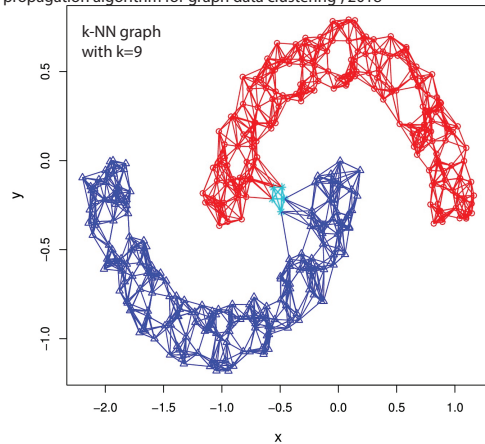
Classic Two Moons Dataset w. Graph Manifold

Graph approximation of two-moons manifold

From: Zhou, Martin, Pan, Liu, "SELP: Semi-supervised evidential label propagation algorithm for graph data clustering", 2018



a. The original data set

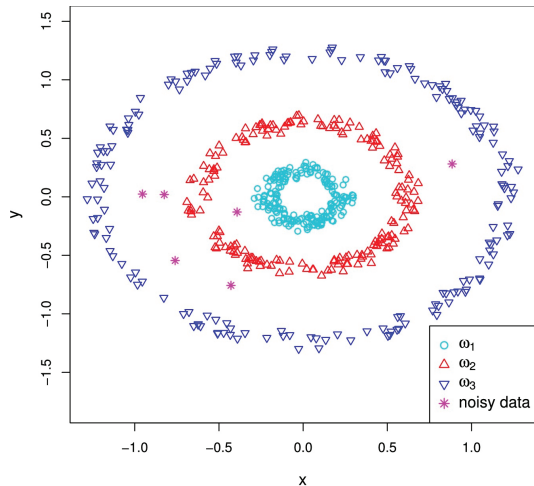


b. The constructed graph

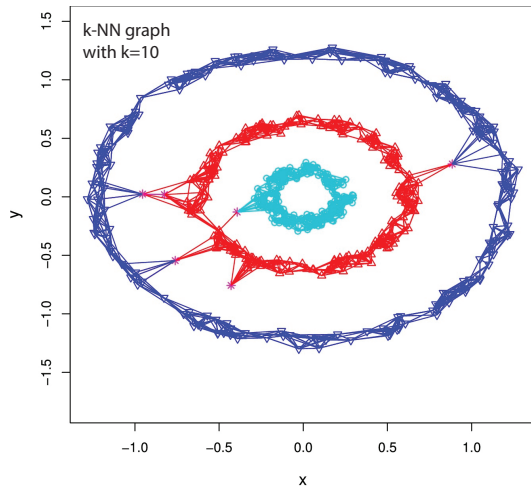
Concentric Rings Graph Manifold

Graph approximation of concentric rings manifold

From: Zhou, Martin, Pan, Liu, "SELP: Semi-supervised evidential label propagation algorithm for graph data clustering", 2018



a. The original data set



b. The constructed graph

Graphical Models

What is Probabilistic Inference?

- Probabilistic Inference involves computing quantities of interest based on probability distributions. E.g., $\forall x_2$, marginalization $\sum_{x_1} p(x_1, x_2)$ or maximization $\max_{x_1} p(x_1, \bar{x}_2)$.
- Probabilistic inference is hard, often NP-hard or even inapproximable.
- Best of cases, exact inference is doable in polynomial time (trees).
- Worst of cases, exact inference is infeasible, so approximation is necessary
- Plethora of approximation methods are possible.
- Probabilistic inference is necessary in many machine learning and AI problems (and is also related to “inference” in other domains, such as sparse joins in relational database systems).
- Probabilistic inference is important in ML since probability theory is (so far) the most successful way to express notions of uncertainty.

Some notation over multi-variate probability distributions

- Distributions

$$p(x) \equiv p(x_{1:N}) \equiv p(x_1, \dots, x_N) \equiv P_{X_1, \dots, X_N}(X_1 = x_1, \dots, X_N = x_N)$$

- Subsets

$$V \triangleq \{1, 2, \dots, N\} \quad A, B \subseteq V \quad A = \{a_1, \dots, a_{|A|}\} \quad (1.2)$$

$$X_A \triangleq \{X_{a_1}, X_{a_2}, \dots, X_{a_{|A|}}\} \quad (1.3)$$

- Example: If $A = \{1, 3, 7\}$ then $X_A = \{X_1, X_3, X_7\}$.
- Example: if $A = \{1, 2\}$ and $B = \{3, 4\}$ then

$$p(X_A = x_A | X_B = x_B) \equiv p(x_1, x_2 | x_3, x_4)$$

- discrete $p(x_A)$ requires table of size $r^{|A|}$, $r_X = |D_X|$ where $\forall i, x_i \in D_X$
- $\bar{x}^{(i)}$ and $\bar{x}^{(j)}$ are different vector-valued samples for $i \neq j$.

What might we want to do with $p(x)$?

- Marginal quantities

- Given $\bar{x}$ compute $p(\bar{x})$
- More generally, given $E \subseteq V$, and $F \cup H = V \setminus E$ with F and H disjoint, then compute for all $x_F \in \mathcal{D}_{X_F}$:

$$p(x_F, \bar{x}_E) = \sum_{x_H} p(x_F, x_H, \bar{x}_E). \quad (1.4)$$

- Model relationship between two signals x_1 and x_2 (e.g., x_1 a feature vector, x_2 is a class or regression variable).
 - compute $p(\bar{x}_1, \bar{x}_2)$.
 - Given $\bar{x}_1$ compute

$$x_2^* \in \operatorname{argmax}_{x_2} p(\bar{x}_1, x_2) \text{ or equivalently } x_2^* \in \operatorname{argmax}_{x_2} p(x_2 | \bar{x}_1) \quad (1.5)$$

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.
- As we mentioned above, we are given set of training samples $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots\}$, and goal is $\theta^* \in \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$ where $L(\theta)$ is overall data loss.

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.
- As we mentioned above, we are given set of training samples $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots\}$, and goal is $\theta^* \in \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$ where $L(\theta)$ is overall data loss.
- Often, want to learn θ^* , where distribution $p_{\theta^*}(x)$ (& marginals $p_{\theta^*}(x_1, x_2)$) is accurate.

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.
- As we mentioned above, we are given set of training samples $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots\}$, and goal is $\theta^* \in \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$ where $L(\theta)$ is overall data loss.
- Often, want to learn θ^* , where distribution $p_{\theta^*}(x)$ (& marginals $p_{\theta^*}(x_1, x_2)$) is accurate.
- Generative learning, cross entropy loss:

$$\mathcal{L}(\theta) = -(1/n) \sum_{j=1}^n \log p_{\theta}(x^{(j)}) + \lambda \|\theta\| \quad (1.6)$$

where $\|\cdot\|$ is some norm. This includes the case of maximum likelihood learning.

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.
- As we mentioned above, we are given set of training samples $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots\}$, and goal is $\theta^* \in \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$ where $L(\theta)$ is overall data loss.
- Often, want to learn θ^* , where distribution $p_{\theta^*}(x)$ (& marginals $p_{\theta^*}(x_1, x_2)$) is accurate.
- Generative learning, cross entropy loss:

$$\mathcal{L}(\theta) = -(1/n) \sum_{j=1}^n \log p_{\theta}(x^{(j)}) + \lambda \|\theta\| \quad (1.6)$$

where $\|\cdot\|$ is some norm. This includes the case of maximum likelihood learning.

- Discriminative learning, cross entropy loss:

$$\mathcal{L}(\theta) = -(1/n) \sum_{j=1}^n \log p_{\theta}(x_2^{(j)} | x_1^{(j)}) + \lambda \|\theta\| \quad (1.7)$$

maximum conditional likelihood learning.

Why might we want to do that with $p(x)$?

- Machine Learning: adjust a model to fit data using maximum likelihood inference.
- As we mentioned above, we are given set of training samples $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots\}$, and goal is $\theta^* \in \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$ where $L(\theta)$ is overall data loss.
- Often, want to learn θ^* , where distribution $p_{\theta^*}(x)$ (& marginals $p_{\theta^*}(x_1, x_2)$) is accurate.
- Generative learning, cross entropy loss:

$$\mathcal{L}(\theta) = -(1/n) \sum_{j=1}^n \log p_{\theta}(x^{(j)}) + \lambda \|\theta\| \quad (1.6)$$

where $\|\cdot\|$ is some norm. This includes the case of maximum likelihood learning.

- Discriminative learning, cross entropy loss:

$$\mathcal{L}(\theta) = -(1/n) \sum_{j=1}^n \log p_{\theta}(x_2^{(j)} | x_1^{(j)}) + \lambda \|\theta\| \quad (1.7)$$

maximum conditional likelihood learning.

- This form of learning almost always requires being able to do “probabilistic inference” efficiently in one form or another.

Learning depends on loss functions, but still needs probabilistic inference

- Alternate discriminative learning, structured **max-margin learning**, might ask for θ to ensure:

$$\log p_{\theta}(x_2^{(j)}, x_1^{(j)}) \geq \log p_{\theta}(x_2, x_1^{(j)}) + \Delta(x_2^{(j)}, x_2), \quad \forall x_2 \quad (1.8)$$

for each sample pair $x_2^{(j)}, x_1^{(j)}$, where $\Delta(x, x')$ measures disparity between x and x' .

- In words, truth is no worse than penalized falsity.
- Encouraged when we minimize

$$\mathcal{L}(\theta) = (1/n) \sum_{j=1}^n \left[\max_{x_2} \left(\log p_{\theta}(x_2, x_1^{(j)}) + \Delta(x_2^{(j)}, x_2) \right) - \log p_{\theta}(x_2^{(j)}, x_1^{(j)}) \right]^+ + \lambda \|\theta\| \quad (1.9)$$

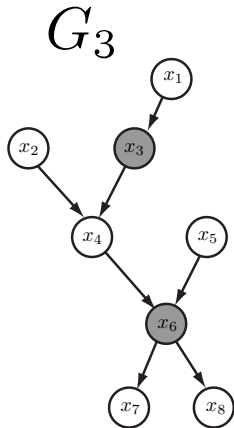
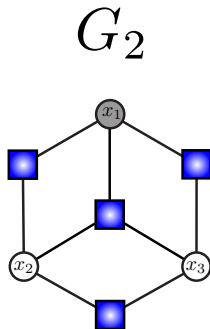
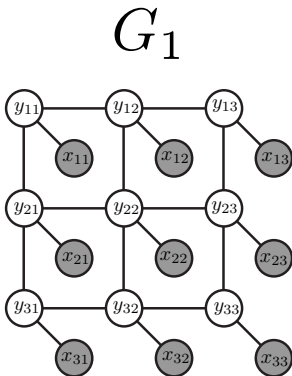
where $\Delta(x_2^{(j)}, x_2)$ is a normalizing mislabeling loss, and $[z]^+ = \max(z, 0)$.

- Generalized hinge-loss, still requires loss-augmented probabilistic inference to solve.

Graphical Models

- A graphical model is a visual, abstract, and mathematically formal description of properties of families of probability distributions (densities, mass functions)

Many types of graphical models (e.g., Markov random fields (left), factor graphs (center), and Bayesian networks (right)):



Graphical Models

- With graphical models, have graph $G = (V, E)$ with $|V| = n$ nodes and we have n random variables one per node.
- For each $v \in V$, X_v is a random variable (r.v.).
- X_V is the complete set of r.v.'s.
- Graphical models are encodings of **families** of probability distributions $p(x_V)$ over X_V .
- Encodings done via a graph that formally specifies either a set (conditional) independence properties or (more fundamentally) a set of factorization properties.
- Crucial idea to understand: Graphical Models encode a set of constraints via factorization requirements/properties that all members of the family must obey.
- Factorization requirements are often but not always identical to conditional independence requirements.
- Factorization, in general, allows sums to be distributed into products thereby making (exact) inference quantities more efficient than if factorization properties did not exist.

Factorization

Definition 1.5.1 (factorization)

Given a function $f(\cdot)$ defined over m variables $x_1, \dots, x_m$, and given $A, B \subseteq [m]$. We say that $f()$ factorizes over x_A and x_B if $\forall x$ there exists functions $g(x_A)$ and $h(x_B)$ such that

$$f(x_A, x_B) = g(x_A)h(x_B). \quad (1.10)$$

In general, A and B may intersect.

- Assume for the moment $A \cap B = \emptyset$.
- Then variables x_a and x_b where $a \in A$ and $b \in B$ do not directly **interact** with each other within f .
- This means, x_b 's influence on the way variable x_a may affect the behavior of f is limited.
- E.g., compare $f(x_a, x_b) = x_a x_b$ (where, no matter the value of x_b , $f(x_a, x_b)$ is still linear in x_a) vs. $f(x_a, x_b) = \log(x_a x_b)$ (where x_b 's value can change the way x_a influences $f()$).

Independence and Conditional Independence

Just a form of factorization

Definition 1.5.2 (independence)

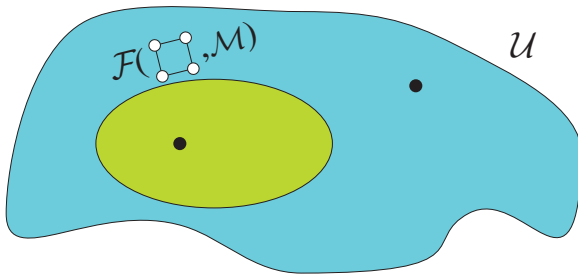
For any probability distribution p over a set of random variables X_V , $V = [m]$, given any collection of two sets $A, B \subseteq V$, we say that X_A is independent of X_B if $p(x_A, x_B) = p(x_A)p(x_B)$ for all possible values x_A, x_B . This is written using the binary conditional independence relation $X_A \perp\!\!\!\perp X_B$.

Definition 1.5.3 (conditional independence)

For any probability distribution p over a set of random variables X_V , $V = [m]$, given any collection of three sets $A, B, C \subseteq V$, we say that X_A is independent of X_B given X_C if $p(x_A, x_B | x_C) = p(x_A | x_C)p(x_B | x_C)$ for all possible values x_A, x_B, x_C . This is written using the ternary conditional independence relation $X_A \perp\!\!\!\perp X_B | X_C$. Whenever $X_A \perp\!\!\!\perp X_B$ (i.e., $C = \emptyset$), we call this *marginal independence* or just *independence as above*.

Markov Properties

- Markov properties are rules that specify what are required of every family member. Any $p \in \mathcal{F}(G, \mathcal{M})$ satisfies all properties/rules $r \in \mathcal{M}$ for G . Any $p \in \mathcal{U} \setminus \mathcal{F}(G, \mathcal{M})$ violates at least one property for G .
- A $p \in \mathcal{U}$ might have more properties. $\mathcal{M}$ is like a filter, lets in those p that satisfy, but will let in those that satisfy more.



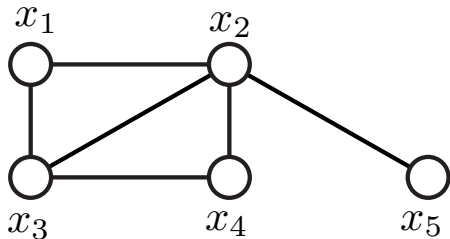
- Example $r \in \mathcal{M}$ might be “if there are two nodes $u, v \in V$ that are neither directly nor indirectly connected in G (i.e., there no path leading from u to v in G) then the corresponding random variables in p are marginally independent”

Questions about Properties: to be mathematically proven

- For a given type of graphical model, can the property set $\mathcal{M}$ be listed in finite space and computed efficiently? (answer, yes).
- For a given type of graphical model, are there more than one set of rules that define a family? In other words, are there distinct rule sets $\mathcal{M}_1$ and $\mathcal{M}_2$ such that $\mathcal{F}(G, \mathcal{M}_1) = \mathcal{F}(G, \mathcal{M}_2)$ for all G ? Answer, yes.
- Much of the Lauritzen 1996 book studies graphs, rules (or Markov properties) and proves when the corresponding families are either identical, or subsets of each other.
- Is there a smallest rule set? In other words, are there rules sets $\mathcal{M}_1 \subset \mathcal{M}_2$ such that $\mathcal{F}(G, \mathcal{M}_1) = \mathcal{F}(G, \mathcal{M}_2)$, and can we compute the smallest set $\mathcal{M}'$ so that $\mathcal{F}(G, \mathcal{M}') = \mathcal{F}(G, \mathcal{M})$ where $|\mathcal{M}'|$ is minimal?
- Are there rule sets that are non-equivalent? I.e. $\mathcal{M}_1$ and $\mathcal{M}_2$ such that $\mathcal{F}(G, \mathcal{M}_1) \neq \mathcal{F}(G, \mathcal{M}_2)$ for some G ? Answer, yes.
- In general, much of graphical model theory is regarding deducing properties of rules and corresponding properties of graphs and the distributions they represent. This allows us to reason about graphs as a means of reasoning about families of distributions.

Clique Factorization in Markov Random Fields

- The “Cliques” of a graph $G = (V, E)$, or $\mathcal{C}(G)$, in a graph are the set of fully connected nodes.
- If $C \in \mathcal{C}(G)$ and $u, v \in C$ then $(u, v) \in E(G)$
- In the following graph



cliques are $\mathcal{C} = \{\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}, \{3, 4\}, \{2, 4\}, \{2, 3, 4\}, \{2, 5\}\}$.

Factorization According to Graph Clique

- Given graph G with cliques $\mathcal{C}(G)$, consider a probability distribution that can be represented as follows:

$$p(x_V) = \frac{1}{Z} \prod_{C \in \mathcal{C}(G)} \phi_C(x_C) \quad (1.11)$$

$$Z = \sum_{x_V} \prod_{C \in \mathcal{C}} \phi_C(x_C) \quad (1.12)$$

- Actually, we don't always need Z explicitly since it is a constant and can be distributed into the factors in a variety of ways, leading to:

$$p(x_V) = \prod_{C \in \mathcal{C}(G)} \phi_C(x_C) \quad (1.13)$$

where only the factorization is depicted.

Graphical Models: A Key Property

- Graphs encode factorization, and factorization makes computation of probabilities much faster (e.g., could go from exponential to linear for, say a chain) for all distributions in a graph's family (amortization).

Graphical Models: A Key Property

- Graphs encode factorization, and factorization makes computation of probabilities much faster (e.g., could go from exponential to linear for, say a chain) for all distributions in a graph's family (amortization).
- Key Problem: there **exists no functions** $g(a)$ and $h(c)$ that constitute a factorization of a sum as in:

$$g(a)h(c) = \sum_b f_1(a, b)f_2(b, c) \quad (1.14)$$

Graphical Models: A Key Property

- Graphs encode factorization, and factorization makes computation of probabilities much faster (e.g., could go from exponential to linear for, say a chain) for all distributions in a graph's family (amortization).
- Key Problem: there **exists no functions** $g(a)$ and $h(c)$ that constitute a factorization of a sum as in:

$$g(a)h(c) = \sum_b f_1(a,b)f_2(b,c) \quad (1.14)$$

- In general, for disjoint variables $A, B, C \subseteq V$, the function

$$f(x_A, x_C) = \sum_{x_B} f_1(x_A, x_B)f_2(x_B, x_C) \quad (1.15)$$

does not factor, exists **no** g, h such that $f(x_A, x_C) = g(x_A)h(x_C)$.

Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b) f_2(b,c)$.

Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b)f_2(b,c)$.
- Approach A: Due to above lack of factorization property, if we summed out b first, it would create an unfactorizable function of the form $h(a,c)$ and would require $O(r^3)$ computation which then would require $O(r^2)$ to get $h(c)$.

Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b)f_2(b,c)$.
- Approach A: Due to above lack of factorization property, if we summed out b first, it would create an unfactorizable function of the form $h(a,c)$ and would require $O(r^3)$ computation which then would require $O(r^2)$ to get $h(c)$.
- Approach B: If, however, we first sum over a , we'd get $f_1(b)f_2(b,c)$ requiring only $O(r^2)$ computation. Then summing over b requires another $O(r^2)$.

Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b)f_2(b,c)$.
- Approach A: Due to above lack of factorization property, if we summed out b first, it would create an unfactorizable function of the form $h(a,c)$ and would require $O(r^3)$ computation which then would require $O(r^2)$ to get $h(c)$.
- Approach B: If, however, we first sum over a , we'd get $f_1(b)f_2(b,c)$ requiring only $O(r^2)$ computation. Then summing over b requires another $O(r^2)$.
- So overall speedup of Approach B over A is $O(r)$.

Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b)f_2(b,c)$.
- Approach A: Due to above lack of factorization property, if we summed out b first, it would create an unfactorizable function of the form $h(a,c)$ and would require $O(r^3)$ computation which then would require $O(r^2)$ to get $h(c)$.
- Approach B: If, however, we first sum over a , we'd get $f_1(b)f_2(b,c)$ requiring only $O(r^2)$ computation. Then summing over b requires another $O(r^2)$.
- So overall speedup of Approach B over A is $O(r)$.
- Key: graphical models help us to guide the computation to avoid these couplings as best as possible (although some coupling is inevitable), leading to exponential speedups.

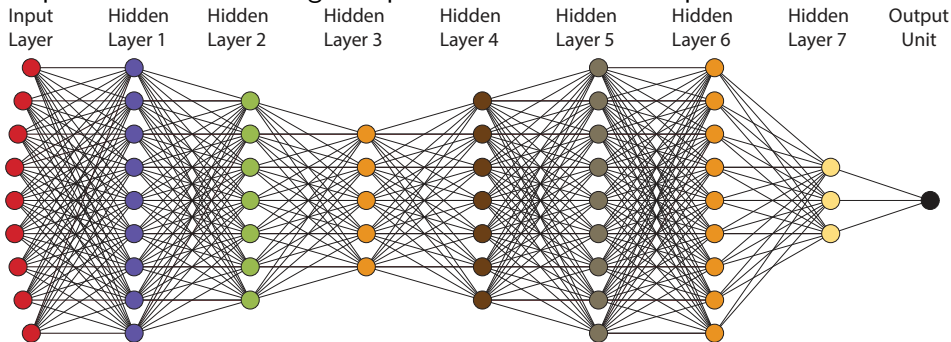
Graphical Models: A Key Benefit

- Suppose we wish to compute $h(c) = \sum_{a,b} f_1(a,b)f_2(b,c)$.
- Approach A: Due to above lack of factorization property, if we summed out b first, it would create an unfactorizable function of the form $h(a,c)$ and would require $O(r^3)$ computation which then would require $O(r^2)$ to get $h(c)$.
- Approach B: If, however, we first sum over a , we'd get $f_1(b)f_2(b,c)$ requiring only $O(r^2)$ computation. Then summing over b requires another $O(r^2)$.
- So overall speedup of Approach B over A is $O(r)$.
- Key: graphical models help us to guide the computation to avoid these couplings as best as possible (although some coupling is inevitable), leading to exponential speedups.
- Core algorithms for doing this involve forms of **message passing on graphs**.

Graph Neural Networks

Graphs can describe (neural) data flow

- Graphs can be used to flow of data through a network.
- Classic example in machine learning. Graphs used to describe deep neural networks.



- Graph shows connectivity pattern, and series of matrix-vector multiplications followed by non-linearities.

Traditional Neural Networks

- Most traditional and much of modern (deep) machine learning is most applicable to grids (e.g., images) or sequences (speech or language).

Traditional Neural Networks

- Most traditional and much of modern (deep) machine learning is most applicable to grids (e.g., images) or sequences (speech or language).
- Many real-world data sources (that can be represented by graphs) are much less regular, no consistent structure, no natural order.

Traditional Neural Networks

- Most traditional and much of modern (deep) machine learning is most applicable to grids (e.g., images) or sequences (speech or language).
- Many real-world data sources (that can be represented by graphs) are much less regular, no consistent structure, no natural order.
- E.g., imagine predicting congestion at a particular time/date of a road network (doing that for all roads in a map).

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.
- Learning becomes:

$$\theta^* \in \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(G^{(i)}, y^{(i)}) \quad (1.16)$$

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.
- Learning becomes:

$$\theta^* \in \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(G^{(i)}, y^{(i)}) \quad (1.16)$$

- Generally, for $i \neq j$, $|V^{(i)}| \neq |V^{(j)}|$, $|E^{(i)}| \neq |E^{(j)}|$, and graphs i and j can be structured entirely differently from each other!

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.
- Learning becomes:

$$\theta^* \in \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(G^{(i)}, y^{(i)}) \quad (1.16)$$

- Generally, for $i \neq j$, $|V^{(i)}| \neq |V^{(j)}|$, $|E^{(i)}| \neq |E^{(j)}|$, and graphs i and j can be structured entirely differently from each other!
- Labels $y^{(i)}$ can be detailed structured properties

GNN Learning Setup

- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.
- Learning becomes:

$$\theta^* \in \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(G^{(i)}, y^{(i)}) \quad (1.16)$$

- Generally, for $i \neq j$, $|V^{(i)}| \neq |V^{(j)}|$, $|E^{(i)}| \neq |E^{(j)}|$, and graphs i and j can be structured entirely differently from each other!
- Labels $y^{(i)}$ can be detailed structured properties
- Prediction can occur at the: (1) node-level, and (2) edge (or link) level, (3) community/group level, or (4) entire graph-level.

GNN Learning Setup

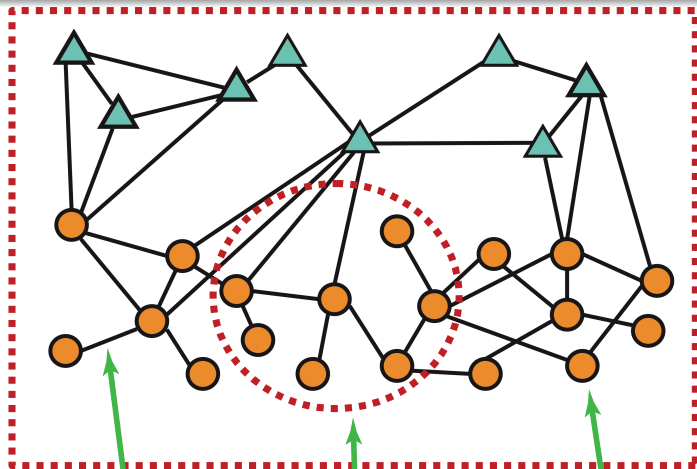
- We again have dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ of n samples, and a loss function $\ell(x, y) \in \mathbb{R}_+$ that expresses badness about predicting x as y , where domain of x (resp. y) is $\mathcal{X}$ (resp. $\mathcal{Y}$).
- Now, $x^{(i)} = G^{(i)}$ where $G^{(i)} = (V^{(i)}, E^{(i)})$ is an input graph and $y^{(i)}$ is some label to predict based on the graph.
- Learning becomes:

$$\theta^* \in \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = (1/n) \sum_{i=1}^n \ell(G^{(i)}, y^{(i)}) \quad (1.16)$$

- Generally, for $i \neq j$, $|V^{(i)}| \neq |V^{(j)}|$, $|E^{(i)}| \neq |E^{(j)}|$, and graphs i and j can be structured entirely differently from each other!
- Labels $y^{(i)}$ can be detailed structured properties
- Prediction can occur at the: (1) node-level, and (2) edge (or link) level, (3) community/group level, or (4) entire graph-level.
- Might wish to predict properties of a node or edge, or predict the existence of a node or edge (e.g., to predict if an edge should or should not exist between two nodes, link-level prediction).

Node, Edge, Graph Prediction in GNNs

Full Graph level
prediction/
generation



Edge level
prediction

Group (subgraph)
prediction

Node level
prediction

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.
- Possible node (intersection) input features: latitude/longitude, type (2-way, 3-way, roundabout), $\exists$ traffic signal type, elevation, POIs (gas station, 7-11), historical traffic,

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.
- Possible node (intersection) input features: latitude/longitude, type (2-way, 3-way, roundabout), $\exists$ traffic signal type, elevation, POIs (gas station, 7-11), historical traffic,
- Possible edge (road) input features: length, speed limit, type (highway, local), num. lanes, one or two-way, surface (paved, gravel, etc.), curvature, accident/incident data, historical traffic data.

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.
- Possible node (intersection) input features: latitude/longitude, type (2-way, 3-way, roundabout), $\exists$ traffic signal type, elevation, POIs (gas station, 7-11), historical traffic,
- Possible edge (road) input features: length, speed limit, type (highway, local), num. lanes, one or two-way, surface (paved, gravel, etc.), curvature, accident/incident data, historical traffic data.
- Dynamic features possible too (time of day, weather, traffic sensors).

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.
- Possible node (intersection) input features: latitude/longitude, type (2-way, 3-way, roundabout), $\exists$ traffic signal type, elevation, POIs (gas station, 7-11), historical traffic,
- Possible edge (road) input features: length, speed limit, type (highway, local), num. lanes, one or two-way, surface (paved, gravel, etc.), curvature, accident/incident data, historical traffic data.
- Dynamic features possible too (time of day, weather, traffic sensors).
- These features might be standardized/embedded (similar to word-2-vec for NLP).

GNN and Feature Embeddings

- Generally, we think of a GNN graph as $G = (V, E, X, W)$ where V is a set of nodes and E a set of edges, but where X and W are **attributes**:
- $X \in \mathbb{R}^{d \times |V|}$ is a set of d -dimensional vectors, one for each node $v \in V$.
- $W \in \mathbb{R}^{d_w \times |E|}$ is a set of d_w -dimensional vectors, one for each edge $e \in E$.
- GNN starts with some input attributes. E.g., in road network, edges are roads and nodes are intersections.
- Possible node (intersection) input features: latitude/longitude, type (2-way, 3-way, roundabout), $\exists$ traffic signal type, elevation, POIs (gas station, 7-11), historical traffic,
- Possible edge (road) input features: length, speed limit, type (highway, local), num. lanes, one or two-way, surface (paved, gravel, etc.), curvature, accident/incident data, historical traffic data.
- Dynamic features possible too (time of day, weather, traffic sensors).
- These features might be standardized/embedded (similar to word-2-vec for NLP).
- We might not always have X and or W .

GNNs and inference

- A GNN can be seen as a multi- (or deep) neural network where each layer is a graph, and going from one layer t to the next layer $t + 1$ corresponds to a form of message passing between nodes of these layers using shared-parameter neural structures.

GNNs and inference

- A GNN can be seen as a multi- (or deep) neural network where each layer is a graph, and going from one layer t to the next layer $t + 1$ corresponds to a form of message passing between nodes of these layers using shared-parameter neural structures.
- Let $h_v^{(t)} \in \mathbb{R}^{d_t}$ be an embedding for each node $v \in V$. At iteration t , we update each embedding $h_v^{(t)}$ via message passing from neighbors embeddings (and also maybe edge embeddings).

GNNs and inference

- A GNN can be seen as a multi- (or deep) neural network where each layer is a graph, and going from one layer t to the next layer $t + 1$ corresponds to a form of message passing between nodes of these layers using shared-parameter neural structures.
- Let $h_v^{(t)} \in \mathbb{R}^{d_t}$ be an embedding for each node $v \in V$. At iteration t , we update each embedding $h_v^{(t)}$ via message passing from neighbors embeddings (and also maybe edge embeddings).
- That is, for all nodes $v \in V$, we at each step (layer) from $t - 1$ to t do:

$$h_v^{(0)} = x_v, \quad \forall v \in V \quad (\text{Initialization}) \quad (1.17)$$

$$m_v^{(t)} = f_{\text{Agg}}^{(t)} \left(\left\{ h_u^{(t-1)}, w(u, v) \mid u \in \mathcal{N}(v) \right\} \right), \quad 1 \leq t < T \quad (\text{Aggregate}) \quad (1.18)$$

$$h_v^{(t)} = f_{\text{Up}}^{(t)} \left(h_v^{(t-1)}, m_v^{(t)} \right) \quad (\text{Update}) \quad (1.19)$$

where $\mathcal{N}(v)$ are the neighbors of v , and final node representation $h_v^{(T)}$ is used for final node/edge/graph prediction (after possible further aggregation and/or simple final linear or MLP classifier). T is considered the depth of the model.

GNNs and message passing

- The aggregation function f_{Agg} enables things like **invariance** and **equivariance** which are often desirable.

GNNs and message passing

- The aggregation function f_{Agg} enables things like **invariance** and **equivariance** which are often desirable.
- Invariance: The output doesn't change when node order or graph structure are permuted (a form of **symmetry**).

GNNs and message passing

- The aggregation function f_{Agg} enables things like **invariance** and **equivariance** which are often desirable.
- Invariance: The output doesn't change when node order or graph structure are permuted (a form of **symmetry**).
- Equivariance: The output changes in a predictable/consistent way under input permutations, preserving structural consistency.

GNNs and message passing

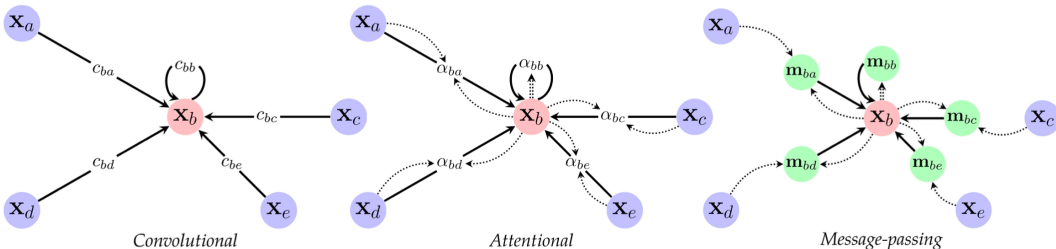
- The aggregation function f_{Agg} enables things like **invariance** and **equivariance** which are often desirable.
- Invariance: The output doesn't change when node order or graph structure are permuted (a form of **symmetry**).
- Equivariance: The output changes in a predictable/consistent way under input permutations, preserving structural consistency.
- The above inference can be seen as a form of **message passing** on graphs.

GNNs and message passing

- The aggregation function f_{Agg} enables things like **invariance** and **equivariance** which are often desirable.
- Invariance: The output doesn't change when node order or graph structure are permuted (a form of **symmetry**).
- Equivariance: The output changes in a predictable/consistent way under input permutations, preserving structural consistency.
- The above inference can be seen as a form of **message passing** on graphs.
- There are different forms of message passing in GNNs, together which encapsulate much of DNNs (e.g., convolutional networks, deep sets, attention networks, transformers, and forms of the symmetries in geometric deep learning).

The three “flavors” of GNN layers

- Most GNN literature is such that g can be classified into one of the following three.



$$\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(\mathbf{x}_j) \right)$$

$$\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} a(\mathbf{x}_i, \mathbf{x}_j) \psi(\mathbf{x}_j) \right)$$

$$\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} \psi(\mathbf{x}_i, \mathbf{x}_j) \right)$$

All Together

Commonality

- The core concept of **message passing** is involved in all three:
- Graph representation of manifolds, message passing for graph-based semi-supervised learning (label propagation, measure propagation).
- Graphical models, message passing algorithms, loopy belief propagation, messages on graphs for coding (e.g., turbo codes, and LDPC codes).
- Graph neural networks, message passing in going from one layer to the next.

The Class Concerto

- ① 25%: Graphs and Graphs in Machine Learning (most generally).
- ② 37.5%: Probabilistic Graphical Models - the use of graphs in graphical models, probabilistic graphical model inference, computing probabilities either exactly or approximately, and applications in machine learning.
- ③ 37.5%: Graph neural networks - using graphs to structure and train deep neural networks.